

16. Un dé, et le hasard du matériel

`hasard()` ne tire pas pile ou face : il lit un compteur qui tourne tout seul.

1. Ce qu'il faut comprendre

`hasard()` rend un octet imprévisible, de 0 à 255 — pas déjà réparti entre 1 et 6. Le reste d'une division, `hasard() % 6`, ramène ça entre 0 et 5 ; il suffit d'ajouter 1 pour une face de dé.

`semer(images())` est appelé une seule fois, au tout début. Sans lui, la console partirait toujours du même point : son horloge interne n'a pas encore tourné à l'instant où le jeu démarre, et deux parties de suite tireraient la même suite.

Le dé ne se relance qu'au front de A, comme la leçon « Se souvenir de l'état d'avant » — sans lui, tenir A referait tourner le dé soixante fois par seconde, bien trop vite pour le lire.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <semer>` → [15.1. La fonction semer\(\) — le départ du hasard](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **851 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          68 o ← la console, à l'allumage - la mise en route de la console
lancerDe()         14 o ← main() - ta fonction
dessinerDe()       19 o ← main() - ta fonction
main()            113 o ← démarrage - ta fonction
VBlank             8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage      28 o ← image() - attend l'image suivante
AttendreVBlank     16 o ← démarrage - attend que l'écran soit libre
AttendreAcces     12 o ← EffacerCases, EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EffacerCases       12 o ← effacer() - efface des cases
EcrireTexte        10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre       33 o ← nombre() - écrit un nombre en chiffres
Reste              10 o ← lancerDe() - le reste d'une division
CopierTuiles       85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage - vide les deux cartes de l'écran
Hasard             25 o ← hasard() - tire un nombre au hasard
LireManette        37 o ← bouton() - lit les huit boutons
Tuiles            320 o ← CopierTuiles - 20 lettres sur 43
"A: LANCER LE DE"  15 o ← texte() - un texte gravé
```

```

"FACE:"          5 o ← texte()  – un texte gravé
"LANCES:"        7 o ← texte()  – un texte gravé
Lettres gravées : ACDEFLNRS0123456789:
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, Diviser, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte()      × 3    33 o sur place, dans main() + EcrireTexte (10 o)
  nombre()     × 2    20 o sur place, dans dessinerDe(), main() + EcrireNombre (33 o)
  bouton()     × 1    12 o sur place, dans main() + LireManette (37 o)
  effacer()    × 1     8 o sur place, dans dessinerDe() + EffacerCases (12 o)
  semer()      × 1     7 o sur place, dans main()
  hasard()     × 1     3 o sur place, dans lancerDe() + Hasard (25 o)
  image()      × 1     3 o sur place, dans main() + AttendreImage (28 o)
  images()     × 1     2 o sur place, dans main()
Tes fonctions appelées :
  dessinerDe() × 2    14 o sur place, dans main()
  lancerDe()  × 1     3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un dé. Chaque appui sur A lance le dé (une face de 1 à 6)
// et compte les lancers.
//
// hasard() rend un nombre imprévisible de 0 à 255.
// hasard() % 6 garde le reste de la division par 6 : de 0 à 5. On ajoute 1 : de 1 à 6.
// Exemple : hasard() rend 200 → 200 % 6 = 2 (car 6 × 33 = 198) → face 3.

#include <hasard>    // tire un nombre au hasard
#include <effacer>   // efface des cases, ou tout le fond
#include <nombre>    // écrit un nombre en chiffres
#include <semer>     // choisit le départ du hasard
#include <texte>     // écrit un texte à l'écran
#include <bouton>    // lit un bouton de la manette
#include <reste>     // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UNE VARIABLE — DE

```

uint8_t de = 1;           // La face affichée (1 à 6).

```

3. UNE VARIABLE — AVANTA

```

uint8_t avantA = 0;       // A était-elle enfoncée au tour d'avant ? (1 ou 0)

```

4. UNE VARIABLE — LANCES

```

uint8_t lances = 0;       // Combien de lancers en tout.

```

5. UNE FONCTION — LANCERDE()

```

uint8_t lancerDe() {      // Une FONCTION qui rend un nombre (uint8_t) :
  return hasard() % 6 + 1; //   « return » donne la réponse : une face de 1 à 6.
}

```

6. UNE FONCTION — DESSINERDE()

```

void dessinerDe(uint8_t face) { // Une fonction qui ne rend rien (void),
                                // et qui reçoit la face à dessiner.
  effacer(8, 6, "0");         // Efface une case (autant d'espaces que de signes : un seul).
  nombre(8, 6, face, 1);      // Écrit la face sur 1 chiffre.
}

```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    semer(images());        // Mélange le hasard avec l'heure de départ, UNE fois.
                            // Sans cela, chaque partie tirerait la même suite de faces.

    texte(2, 2, "A: LANCER LE DE");
    texte(2, 4, "FACE:");
    texte(2, 9, "LANCES:");
    dessinerDe(de);         // La face de départ : 1.

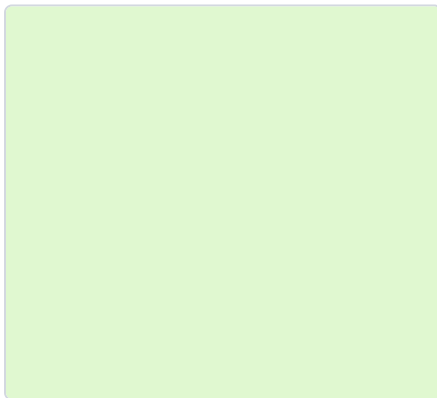
    while (true) {          // La boucle du jeu :
        uint8_t a = bouton(A); // A enfoncée maintenant ? 1 ou 0.

        if (a && !avantA) {  // Enfoncée maintenant ET pas au tour d'avant :
            // c'est le DÉBUT de l'appui (on dit « le front »).
            de = lancerDe();  // on lance,
            dessinerDe(de);   // on dessine la face,
            lances++;         // un lancer de plus,
            nombre(10, 9, lances); // on l'affiche.
        }
        avantA = a;         // On retient A pour le tour suivant.

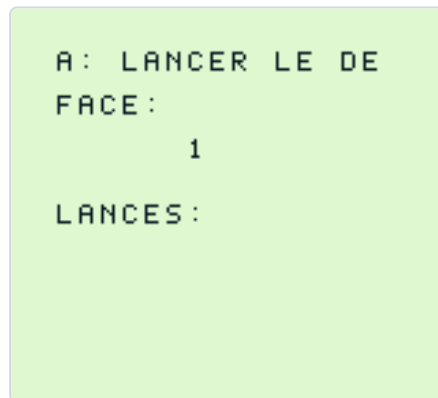
        image();            // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

```
A: LANCER LE DE
FACE:
    2
LANCES: 001
```

A — l'écran a changé

5. Ce qu'on doit voir

Un dé de 1 à 6 sous FACE, qui change à chaque appui sur A ; LANCES compte les appuis.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ un dé de départ, entre 1 et 6 (1)
- ✓ chaque lancer reste entre 1 et 6 (1, 6, 6, 1, 1, 1, 1, 6)
- ✓ huit lancers ne rendent pas tous la même face (2 valeurs différentes)
- ✓ le compteur de lancers suit (8)

Ensuite : **17. Deux mots de même longueur** — chapitre 3 : Retenir, et réagir. · `gameboy3` — écrire une cartouche Game Boy en C++